

Manual Técnico do Script `credenciais_processor.py` - Sistema Hórus

Versão do Documento: 1.0

Data de Atualização: Dezembro de 2025

Tecnologia Principal: Python 3 + PostgreSQL

Este manual técnico descreve em detalhes o script `credenciais_processor.py`, que atua como o motor de processamento de credenciais vazadas no sistema Hórus. O script é responsável por validar, extrair, normalizar e inserir dados de credenciais em arquivos de entrada, garantindo deduplicação e foco em domínios relevantes (militares/governamentais). Ele integra com o banco de dados PostgreSQL e gera relatórios HTML para visualização.

O documento explica as funcionalidades principais, configurações, funções disponíveis, restrições de uso e mecanismos de validação implementados. Ele é destinado a administradores, desenvolvedores e mantenedores que precisam compreender ou modificar o comportamento do processamento de dados.

1. Introdução e Propósito

O `credenciais_processor.py` é o componente central de ETL (Extract, Transform, Load) do Hórus, uma plataforma de Threat Intelligence para detecção e análise de credenciais vazadas. Ele processa arquivos de entrada (.txt, .xls, .xlsx), valida formatos de usuário (CPF, e-mail, nome), senhas e sites, gera hashes para deduplicação e insere registros no banco de dados. Além disso, suporta integração com fontes externas como Zimbra e gera relatórios HTML para dashboards e visualizações.

O script utiliza regex pré-compiladas para eficiência, Pandas para manipulação de Excel e psycopg2 para interações com PostgreSQL. Ele opera de forma autônoma via linha de comando ou integrado a outros scripts (ex: `app.py` para uploads web).

1.1 Dependências e Integrações

- **Linguagem:** Python 3.
- **Banco de Dados:** PostgreSQL (via psycopg2), com suporte a conexões contextuais e execução de queries.
- **Bibliotecas:** Pandas (para Excel), re (regex), hashlib (hashes SHA1/MD5), urllib.parse (parsing de URLs), Matplotlib (gráficos em relatórios).
- **Integrações Externas:** Banco de dados repo (tabelas credenciais e credenciais_geral), Zimbra (via processar_lista_externa).
- **Scripts Relacionados:** Chamado por app.py (uploads), zimbra_parser.py (extração ML), password_analyzer.py (análise de senhas).

2. Funcionalidades Principais

O `credenciais_processor.py` fornece um pipeline robusto para processamento de credenciais, com foco em validação rigorosa e eficiência. Abaixo, as principais funcionalidades:

- **Validação de Dados:** Verifica CPFs (com dígitos verificadores), e-mails (regex), nomes (suporte a acentos/especiais) e sites (allowlist de domínios .mil.br, .intraer, etc.).
- **Extração e Parsing:** Processa linhas de arquivos texto/Excel, identifica campos (usuário, senha, site) com prioridade para CPF/e-mail, parseia URLs e ignora linhas inválidas (ex: caracteres não ASCII, campos insuficientes).
- **Deduplicação:** Gera hashes SHA1 para senhas e combinações usuário+senha, insere com ON CONFLICT para atualizar metadados sem duplicatas.
- **Inserção no Banco:** Batch insertion em tabelas credenciais (únicas) e credenciais_geral (histórico), com suporte a source_bttnng para rastreio de origem.
- **Geração de Relatórios:** Cria arquivos HTML para relatórios únicos/gerais, dashboards com gráficos (ex: distribuição mensal) e arquivos de processados/rejeitados.
- **Integração com Zimbra:** Função dedicada para processar listas externas de credenciais extraídas por ML.
- **Tratamento de Arquivos:** Suporte a .txt (linhas delimitadas por :/|/;), Excel (colunas username/email, password, url), com formatação automática de CPFs.
- **Busca de Usuários:** Função para busca parcial (ILIKE) em credenciais, retornando registros formatados.
- **Geração de Gráficos:** Usa Matplotlib para gráficos em base64 nos dashboards (ex: distribuição de inserções).

3. Configurações

O script define configurações globais e de ambiente via imports de config.py e variáveis hardcoded:

- **Diretórios e Arquivos:** DIR_RESULTADOS='resultados', ARQUIVO_PROCESSADOS='processados.txt', ARQUIVO_REJEITADOS='rejeitados.txt', ARQUIVO_HTML='web.html', ARQUIVO_DASHBOARD='dashboard.html'.
- **Codificação:** CODIFICACAO='utf-8' para leitura/escrita de arquivos.
- **Globais Temporais:** CURRENT_TIME e CURRENT_USER para timestamps em relatórios.
- **Regex Pré-Compiladas:** REGEX_CPF, REGEX_EMAIL, REGEX_NOME, REGEX_CARACTERES_VALIDOS, REGEX_URL para validações otimizadas.
- **Domínios Permitidos:** Lista em parse_site (ex: '.mil.br', 'decea.gov.br').
- **Banco de Dados:** Usa DB_CONFIG_REPO e get_active_db_config() para conexões; autocommit falso.
- **Outras:** Extensões permitidas em process_file (.txt, .xls, .xlsx); suporte a errors='ignore' em leituras.

Restrições: Não instala pacotes (sem internet); paths hardcoded (ex: /Dados/HIBP para HIBP, mas não usado aqui).

4. Funções e Métodos Principais

O script define funções modulares para cada etapa do pipeline. Cada função é descrita abaixo, incluindo parâmetros, funcionalidades e restrições:

- **validar_cpf(cpf)**: Valida CPF com dígitos verificadores. Retorna bool. Restrição: Remove não-dígitos antes de validar.
- **formatar_cpf(cpf)**: Formata CPF como XXX.XXX.XXX-XX.
- **validar_email(email)**: Valida e-mail com regex. Retorna bool.
- **validar_nome(nome)**: Valida nomes com suporte a especiais. Retorna bool.
- **validar_usuario(user)**: Wrapper para validar CPF/email/nome.
- **parse_site(url_part)**: Parseia e valida sites contra allowlist. Retorna hostname ou None.
- **determine_user_password(parts)**: Identifica usuário/senha com prioridade CPF/email.
- **process_line(original_line)**: Processa linha individual: split, valida campos, identifica user/pass/site. Retorna dict com 'valid' e 'reason'. Restrição: Exige 2-3 campos; ignora linhas vazias/inválidas.
- **get_repo_connection()**: Context manager para conexão PostgreSQL (repo).
- **execute_repo_query(query, params, ...)**: Executa queries com fetch options e commit.
- **inserir_no_banco(credenciais_validas)**: Insere batch em credenciais_geral e credenciais com ON CONFLICT. Retorna novos, duplicados, falhas.
- **gerar_html()** / **gerar_html_geral()**: Gera HTML para relatórios únicos/gerais de credenciais.
- **gerar_estatisticas()**: Gera relatório de estatísticas (totais, comuns) em HTML.
- **gerar_dashboard()**: Gera dashboard HTML com gráficos e estatísticas.
- **buscar_usuario(user)**: Busca parcial em credenciais, retorna lista de dicts.
- **process_file(filepath, result_folder)**: Pipeline principal: lê arquivo, processa linhas, insere no banco, gera relatórios. Suporta .txt/Excel. Retorna novos/duplicados.
- **processar_lista_externa(lista_dados, filename)**: Processa listas de dicts (ex: de Zimbra), gera hashes e insere. Retorna stats.
- **get_html_content(filename)**: Lê HTML com tratamento de erros.

Se executado como main: Parser de argumentos para arquivo de entrada e pasta de resultados.

5. Restrições e Segurança

- **Formatos de Entrada:** Apenas .txt (delimitado), .xls/.xlsx (colunas específicas). Ignora linhas inválidas, anexa rejeições em 'rejeitados.txt'.
- **Validações:** Rígidas para domínios (allowlist); ignora sites não militares. CPFs/e-mails priorizados; nomes genéricos como fallback.
- **Deduplicação:** Baseada em hashes; atualiza data_insercao em conflitos.
- **Segurança:** Hashes SHA1 para senhas (não reversíveis); conexões com rollback em erros. Sem exposição de senhas claras em logs/relatórios.
- **Performance:** Batch insertion para grandes volumes; regex pré-compiladas. Restrição: Não paralelizado.
- **Erros:** Tratados com prints e raises; rejeições logadas em arquivos append.
- **Restrições Gerais:** Foco em domínios .br; sem suporte a outros formatos (ex: JSON). Paths hardcoded (ajuste em config.py).

Nota Final

O `credenciais_processor.py` é modular e eficiente, mas alterações devem preservar validações e conformidade (ex: LGPD para dados sensíveis). Consulte o código para extensões ou integre com ML para parsing avançado. Em produção, monitore logs de rejeições para tuning de regex.